॥श्रीगणेशाय नमः॥

Operating Systems

An OS is an interface between the user and the hardware. An OS manages all the resources of a computer (CPU, memory, files, devices).

Types of operating systems

i. Batch OS:

We have a batch of jobs. All are submitted at once. After a long time, the user collects the output. These were used in the 80's.
Jobs are executed sequentially. One job finishes at a time.



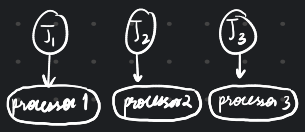
ii. Multiprogramming or multitasking OS:

All the jobs will share the CPU. Every job will get a chance frequently.
Interactivity is more. The CPU switches between the processes.



iii. Multiprocessing OS:

There'll be many processors. So many jobs can be executed in parallel.



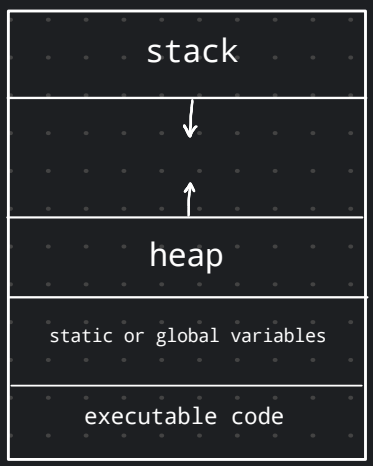
iv. Real-time OS:

Jobs will have deadlines.

Process:

A process is the instance of a computer program that is being executed by 1/more threads.

On running a process, a data structure is created in the main memory.



mainly used for function calling and local variables

mainly used for dynamic memory allocation

Scheduler: A part of the OS that takes a program from the second memory, converts it into a process, and puts it into the main memory.

Attributes of a process:

i. Process id: a unique # given to a process. (similar to a port #)

ii. Program counter: the next instruction to be executed. Say p1 is executing, PC = 10. Say it's preempted. When it returns, it'll continue from instruction # 10.

iii. Process state: Every process has a state at any time.

iv. Priority: Though processes usually have one, a priority isn't mandatory.

v. General purpose registers: Store intermediate values generated in execution, to be restored once a process comes back after preemption.

vi. List of open files

vii. List of open devices

viii. Protection

All these attributes are stored in a data structure called the process control block (PCB).

Context switching:
storing register values in the PCB of the process to be preempted
and bringing in new register values from another process' PCB

> In computing, a **context switch** is the process of storing the state of a process or thread, so that it can be restored and resume execution at a later point, and then restoring a different, previously saved, state.[1] This allows multiple processes to

Every process will be managed by storing all information in the PCB.

The OS will maintain a linked list of PCB's of all processes.
The OS will look into the PCB's and decide which process to give the CPU to.

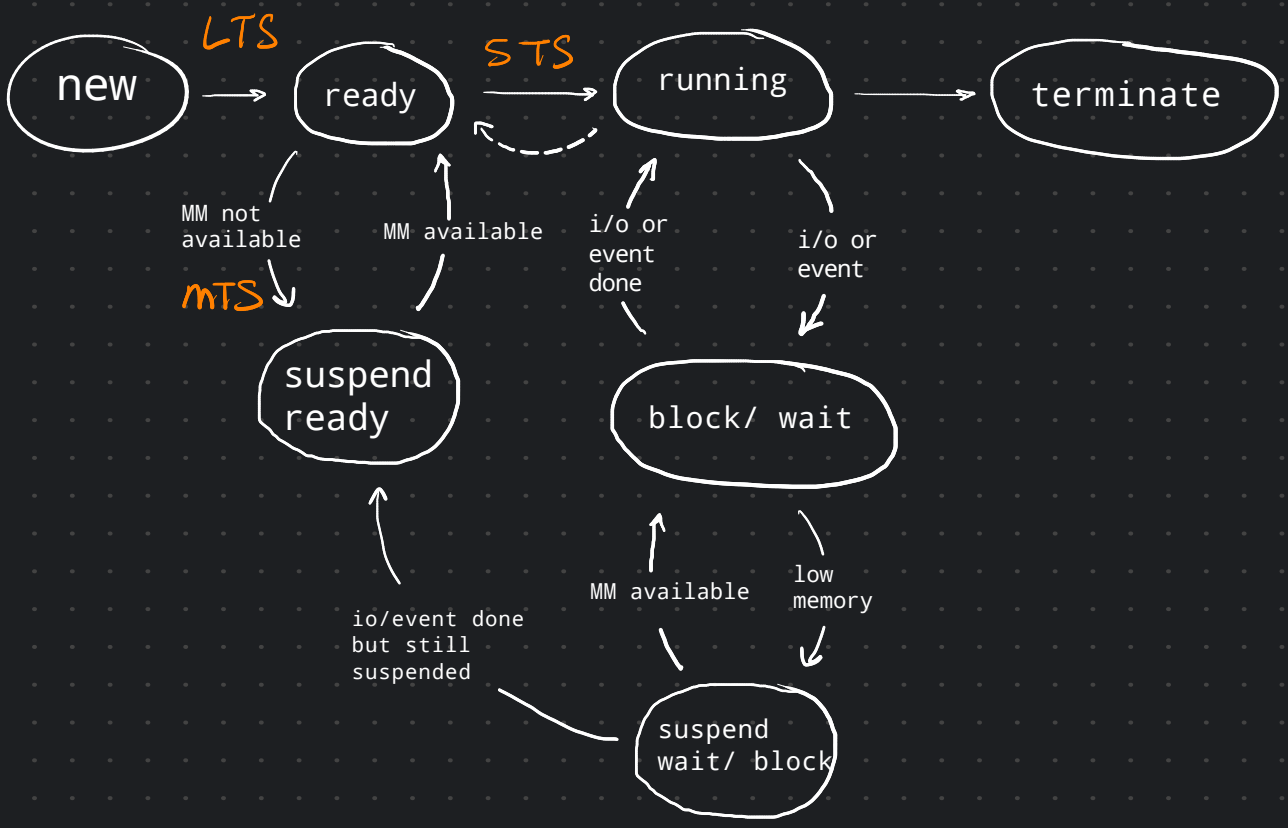
A PCB is also called a context.

Preemption: forcefully remove a process during its execution.

States of a process

i. new state: the process is about to be created; the program is still in the secondary memory;
ii. ready state: the process is created, is in the main memory, and is ready to be executed;
many processes can be in the new or ready states;
iii. running state: the process is executing in the CPU; if we have 1 CPU, then a maximum of 1 process can be in the running state;
iv. block/ wait: a process may need i/o or may get blocked due to some event;
v. termination/ completion: the process is killed; the context/ PCB is deleted;
vi. suspend ready: if there's no more space in the main memory, one of the processes has to be moved to the secondary memory to create more space;
the process will move to suspend ready from ready;
vii. suspend wait/ block:

A process can complete i/o when in the blocked state.

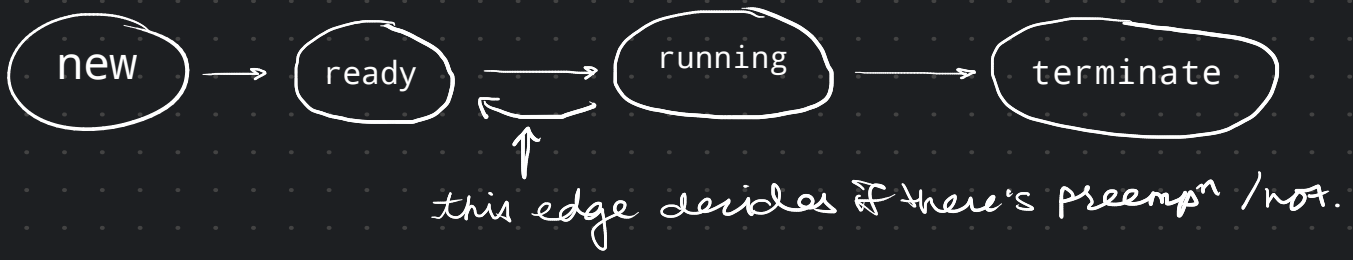


Types of schedulers:

i. Long-term scheduler: creating a process
ii. Short-term scheduler: scheduling from ready to running
iii. Medium-term scheduler: suspending a process



Why does a process get blocked?
- waiting for i/o
- some event (semaphores)



Degree of Multiprogramming:
maximum # of processes that can be present in the ready state at a time.

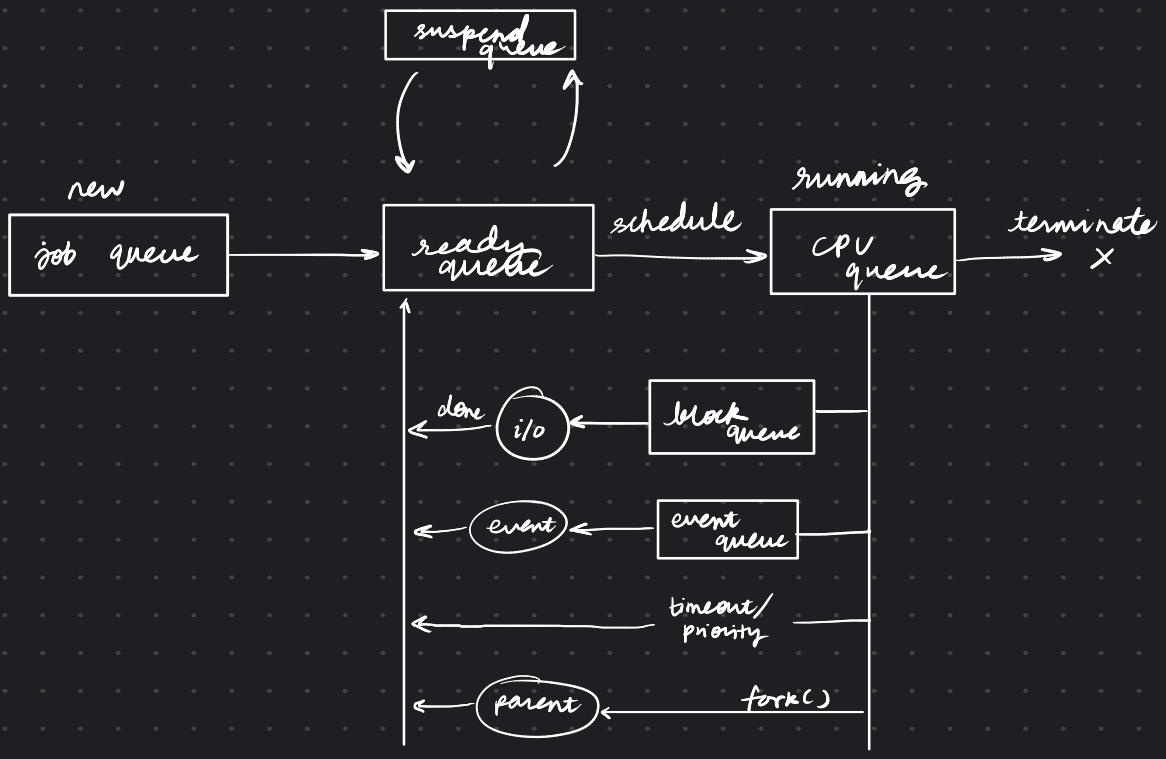
(decided by LTS)

Gate: Consider a system with 'N' CPUs and 'm' processes, then what is the max and min in the following.

	min	max.
✓ ready	0	m
✓ running	0	N
block	0	m

- N CPUs
- m processes

Scheduling queues or process queues:



Scheduling algorithms

- Arrival time:
- Completion time:
- Turn around time:
- Burst time:
- Waiting time:
- Response time:

In non-preemptive algorithms, response time = waiting time

CPU Scheduling:

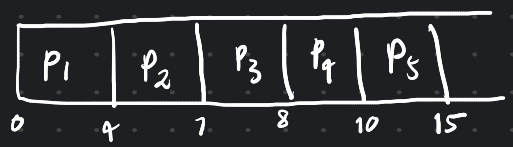
picking up a process from ready state and giving it to the CPU.
STS + dispatcher do it

When will STS do the scheduling?
running -> termination (when a process ends, a new one is scheduled)
running -> wait (when a process gets blocked)
running -> ready (when a process is preempted)
new -> ready (when a process is newly created, it may be eligible for immediate scheduling)
wait -> ready (when a process becomes eligible)

First come, first serve:

criterion: arrival time
mode: non-preemptive

P NO	AT	BT	CT	TAT	WT	RT
1	0	4	4	4	0	0
2	1	3	7	6	3	3
3	2	1	8	6	5	5
4	3	2	10	7	5	5
5	4	5	15	11	6	6



length of schedule = starting of 1st process to end of last process

here 15.

Adv:
simple to implement; queue is used

Disadv:
convoy effect: a large process might make others wait for long (starvation)

If arrival time is same, pid will be used as the tiebreaker. (choose the lesser)

Ex on Convoy effect.

when ever AT is same, use smaller process number

PNO	AT	BT	CT	TAT	WT
1	0	20	20	20	0
2	1	2	22	21	19
3	1	1	23	22	21

AV TT = 21

A WT = 40/3 = 13.33

Convoy effect

AV TAT = 9

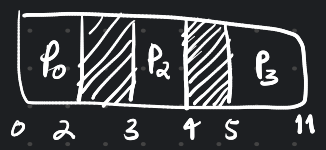
AV WT = 4/3 = 1.3

PNO	AT	BT	CT	TT	WT
1	1	20	23	22	2
2	0	2	2	2	0
3	0	1	3	3	2

bsanved@gmail.com

PNO	AT	BT
1	0	2
2	3	1
3	5	6

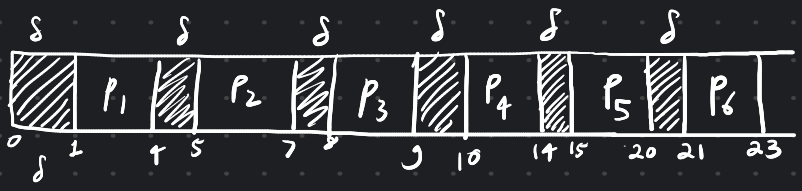
CT	TT	WT
2	2	0
4	1	0
11	6	0
	Avg = 3	Avg = 0



FCFS with Context switching overhead. Let Context Switching time δ = 1 unit

bsanved@gmail.com
7798391468

PNO	AT	BT
1	0	3
2	1	2
3	2	1
4	3	4
5	4	5
6	5	2



total time = 23, of which 6 units is content switches.

$$\text{Inefficiency} = \frac{6}{23} \times 100\ \%$$

$$\therefore \text{efficiency} = \frac{17}{23} \times 100\ \%$$

SJF: Shortest job first

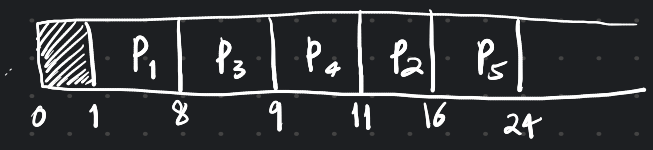
Out of all processes available in ready state, we'll be schedule the process with the shortest burst time.
Criterion: burst time
Mode: non-preemptive



PNO	AT	BT	CT	TAT	WT
1	1	7	8	7	0
2	2	5	16	14	9
3	3	1	9	6	5
4	4	2	11	7	5
5	5	8	24	19	11

avg = $\frac{53}{5}$ = 10.6 avg = 6

when BT is same,
lower AT will take precedence.
If that's same as well,
then lower pid

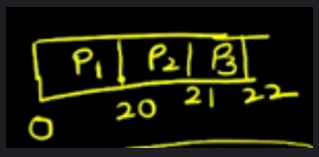


SJF also has convoy effect.

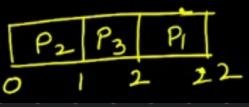
PNO	AT	BT	CT	TT	WT
1	0	20	20	20	0
2	1	1	21	20	19
3	2	1	22	20	19

avg WT = $\frac{38}{3}$ = 12.66

high as the big process arrives first.



PNO	AT	BT	CT	TT	WT
1	2	20	22	20	0
2	0	1	1	1	0
3	1	1	2	1	0



here avg WT = 0 as the big process arrived later.

Preemption can solve convoy effect.

SJF is based on BT, which can't be known beforehand. So it's impractical. Often it is used to benchmark performance of other scheduling algos.

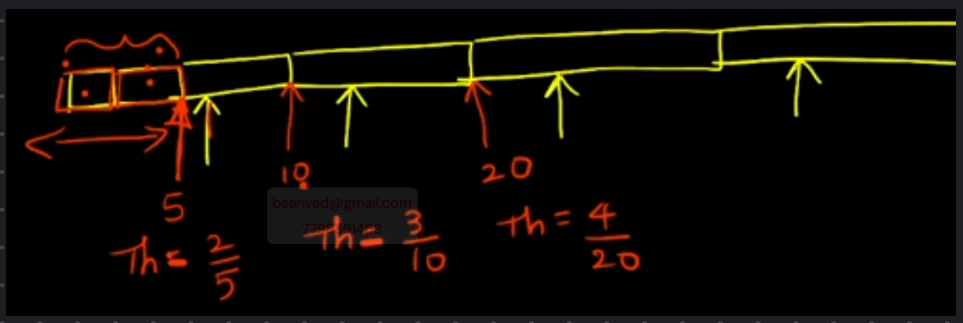
Any algo based on BT can't be practically implemented.

The BT could be predicted, even if not exactly.

Throughput = # of processes completed per second = total # of processes / schedule length

All scheduling algos will have the same throughput at the end.

Throughput of SJF is higher in the middle/ at any point during execution.



lly, avg TAT and avg WT of SJF is less (except for the convoy effect case)

So SJF is the best algo, but it's useless, since we don't know the BT.

Prediction Techniques

Static:	Dynamic:
- process size	- simple averaging
- process type	- exponential averaging

i. Process size:
we'll guess the BT based on past experience. E.g. say in the past,
a 200 KB process took 20 units time, then a process of 201 KB would take approx 20 mins as well.
Problem: It is inaccurate, as the 200 KB process may contain 10 loops, the 201 kb one may contain 1000.

ii. Process type:
We classify processes into types and state BT for each time.
The processes could be OS processes (system calls), user processes (interactive, like games, foreground, like MS Word, or background, like a music player).
We'll say something like the OS processes would take 1 unit, interactive ones would take 2 units, foreground would take 3 units, or something else.
But this is inaccurate as well.

iii. Simple averaging:
Say we know the burst time of the n previously executed processes.
The BT of the n+1 th process would be the average of the previous n processes.
This could be inaccurate.

iv. Exponential averaging:

$$\tau_{n+1} = \alpha t_n + (1-\alpha)\tau_n$$

t_n → actual BT of n

τ_n → predicted BT of n

α → smoothening factor

$\alpha \in [0,1]$

Q. Gate: $\alpha = 0.5$, $\tau_1 = 10$, actual BT $(t_1, t_2, t_3, t_4) = (4, 8, 6, 7)$, $\tau_5 = ?$

→

$\tau_2 = (0.5)4 + (0.5)(10) = 7$

$\tau_3 = (0.5)8 + (0.5)7 = 7.5$

$\tau_4 = (0.4)6 + (0.5)(7.5) = 6.75$

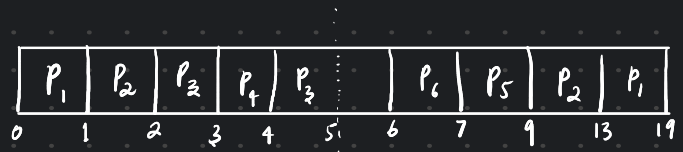
$\tau_5 = (0.5)7 + (0.5)(6.75) = 6.875$

SRTF: shortest remaining time first

preemptive version of SJF
Criterion: BT (if same, then lower AT first)
Mode: preemptive

Q.

PNO	AT	BT
1	0	7
2	1	5
3	2	3
4	3	1
5	4	2
6	5	1



at this time, all the processes have arrived. Now it acts like SJF.

Q.

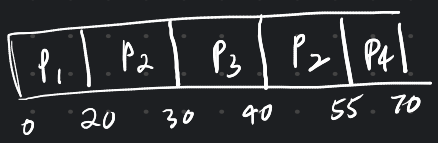
PNO	AT	BT	CT	TAT	WT
1	0	9	13	13	4
2	1	4	5	4	0
3	2	9	22	20	11

Avg. WT = $\frac{15}{3} = 5$



Q.

PNO	AT	BT	CT	TAT	WT
1	0	20	20	20	0
2	15	25	55	40	15
3	30	10	40	10	0
4	45	15	70	25	10



HRRN: Highest Response Ratio Next

SJF and SRTF are the best algos; It's just that long jobs wait longer. So there's starvation.

criterion: response ratio = (WT + BT) / BT = 1 + WT/BT

$$RR = 1 + \underbrace{\frac{WT}{BT}}_{\uparrow}$$

this needs to be high

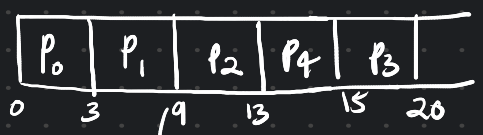
BT ↓ RR ↑ ⇒ shorter jobs get priority

WT ↑ RR ↑ ⇒ if a process waits too long, it'll get selected.

So shorter jobs get selected, and longer jobs get selected on waiting a while as well.

Q. (non-preemptive)

PNO	AT	BT
0	0	3
1	2	6
2	4	4
3	6	5
4	8	2



at this point, P_2, P_3, P_4 are available. (t = 9)

$RR_2 = 1 + \frac{5}{4}$ $RR_3 = 1 + \frac{3}{5}$ $RR_4 = 1 + \frac{1}{2}$

highest

so P_2 will be scheduled.

Now at t = 13,

$RR_3 = 1 + \frac{7}{5}$ $RR_4 = 1 + \frac{5}{2}$

higher

so P_4

comparing to SJF,

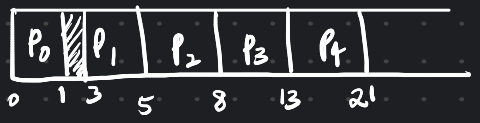


Here P_2 is postponed.

In HRRN, P_2 is selected even though it's longer as it has waited enough.

HRRN

PNU	AT	BT	CT	TAT
0	0	1	1	1
1	3	2	5	2
2	5	3	8	3
3	6	5	13	7
4	7	8	21	14



At $t=8$, $RR_3 = 1 + \frac{2}{5}$ higher $RR_4 = 1 + \frac{1}{8}$

---

best throughput, avg WT, avg TAT -> SJF
practical -> RR

Priority Scheduling



criterion: priority
mode: priority scheduling could be preemptive or non-preemptive

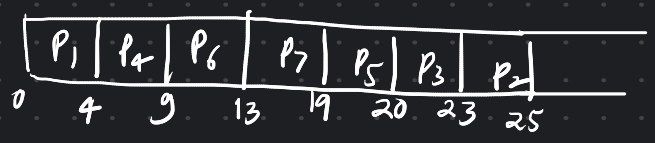
either the lower, or the higher priority # could be considered of higher priority value (it'll be explicitly specified)

tiebreaker: arrival time, then pid

(H)

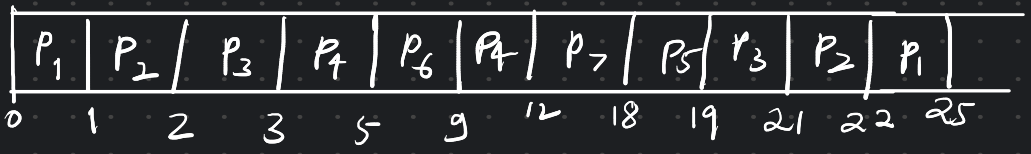
Non preemptive Priority Scheduling

PNO	Priority	AT	BT
1	2 (L)	0	4
2	4	1	2
3	6	2	3
4	10	3	5
5	8	4	1
6	12	5	4
7	9	6	6



Gate: Preemptive priority:

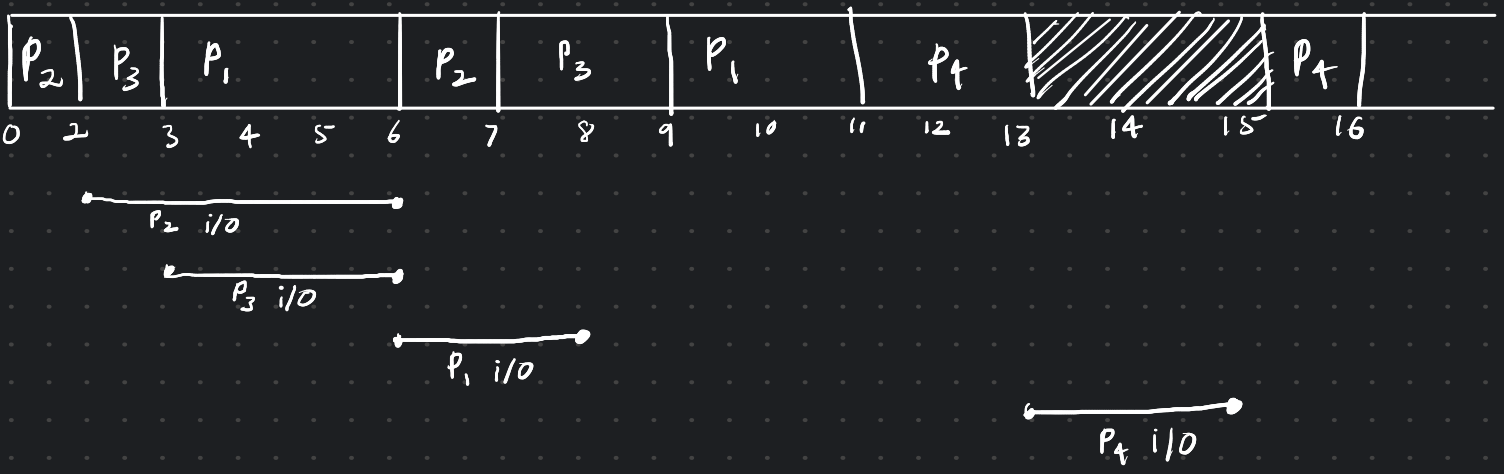
PNO	Pri	AT	BT
1	2 (L)	0	4
2	4	1	2
3	6	2	3
4	10	3	5
5	8	4	1
6	12 (H)	5	4
7	9	6	6



SRTF with processes containing CPU and i/o time:

SRTF:

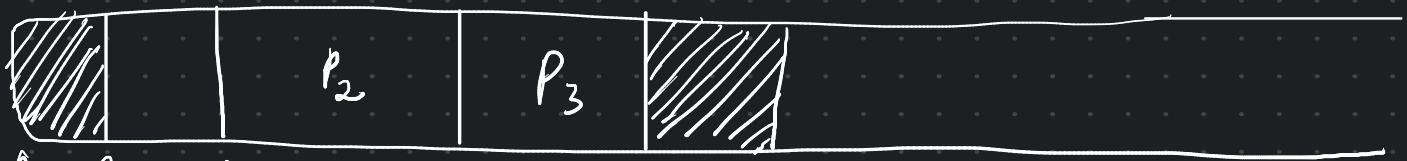
PNO	AT	BT	I/O BT	BT
1	0	3	2	2
2	0	2	4	1
3	2	1	3	2
4	5	2	2	1



Gate: Consider 3 processes, all arriving at time zero, with total execution time of 10, 20 and 30 units respectively. Each process spends the first 20% of execution time doing I/O, the next 70% time doing computation and the last 10% time doing I/O again. The OS uses a SRTF algo and schedules a new process either when the running process gets blocked on I/O or when the running process finishes its compute burst. Assume all I/O operations can be overlapped as much as possible. Find what % of time does CPU remain idle.

→

	i/o	BT	i/o
P_1	2	7	1
P_2	4	14	2
P_3	6	21	3



P_1 —— P_1 ——

P_2 —— P_2 ——

P_3 —— P_3 ——

idle time = 5

total time = 47

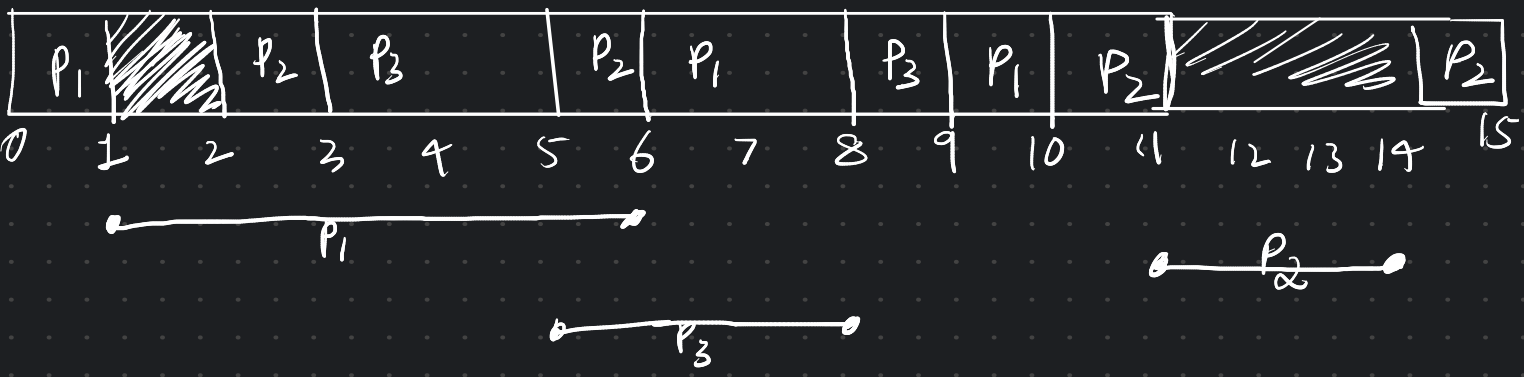
$$\%\ \text{idle time} = \frac{5}{47} \times 100$$

$$= 10.6382\ \%$$

Preemptive priority:

PNO	AT	Pri	BT	I/O	BT
1	0	2	1	5	3
2	2	3 (L)	3	3	1
3	3	1 (H)	2	3	1

completion time of P_2?

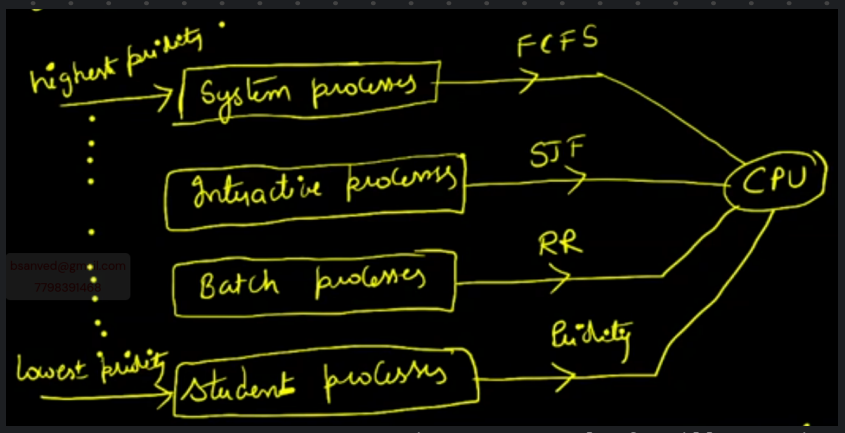


→ 15

$\eta = \frac{11}{15} = 73.33\%$

Multilevel queue scheduling:

In some OS, a MLQ is used.



just an example for illustration

Problem: Starvation at the lower levels.

